Guia de onboarding para desenvolvedores: projeto Mut

Versão de 12/10/2026, branch `feat/admissao-verificada-sdd`.

Este documento é para quem está entrando no projeto agora. Ele explica o negócio, o repositório, como rodar tudo, como cada motor funciona e, com mais profundidade, o motor da AVAP, que é a frente mais ativa. Foi escrito a partir do código e dos docs do repositório. Quando o código e algum doc antigo divergem, vale o código, e as divergências conhecidas estão marcadas. Para as **regras de negócio da AVAP**, a referência é `docs/REGRAS_RATEIO_AVAP.md`, a fonte única.

Como ler: se você tem pouco tempo, leia as seções 1, 2 e 3, depois a 8 inteira, depois a 11. O resto é consulta. Antes de mexer na AVAP, leia também `docs/REGRAS_RATEIO_AVAP.md`.

Sumário

1. O projeto em uma página
2. Contexto de negócio e regulatório
3. Mapa do repositório
4. Colocando o ambiente de pé
5. Arquitetura e princípios
6. Motor ELLO
7. Motor AutoTruck
8. Motor AVAP (foco deste guia)
9. Admissão Verificada (MVP em SDD)
10. Material comercial e de estratégia
11. Estado atual, pendências e próximos passos
12. Convenções de trabalho
13. Glossário
14. Onde ler mais

---

1. O projeto em uma página

O Mut é uma plataforma de **rateio e faturamento** para associações de proteção veicular. Nessas associações, os associados dividem entre si o custo dos eventos (colisão, roubo, furto, perda total). Todo mês alguém precisa responder: quanto cada veículo paga, e por quê. Hoje isso é feito em planilha, com muita decisão manual e pouca rastreabilidade.

O projeto transforma esse fechamento mensal em um cálculo determinístico, auditável e reprodutível, com prova de que fecha ao centavo.

Há três "verticais", uma para cada associação cliente, cada uma com regra de rateio diferente:

Vertical	Associação	Modelo	Situação
ELLO	seguro de moto	valor fixo por cota; os eventos preenchem "potinhos" até o alvo do mês	motor pronto; dados reais chegaram, falta base de veículos
AutoTruck	veículos leves, motos e pesados	repasse de custo dividido por cotas, por segmento	motor aprovado com dado real (Δ = R$ 0,00)
AVAP	seguro de carro	repasse de custo dos eventos escolhidos, mais linhas de soma e dedução, dividido por cotas em pote único	fases 0, 1 e 2 prontas; setembro aprovado com Δ = 0; tela de demonstração pronta

As três associações atuam sob a **Mutualiza**, a administradora que a LC 213/2025 passou a exigir. A Mutualiza é a ponte comercial e a fonte dos dados reais.

Duas ideias atravessam tudo e você precisa internalizar logo:

1. **Aceite é "Δ = 0 contra o fechamento real".** Um motor só é considerado pronto quando reproduz, com diferença zero, o fechamento que a associação fez na mão. Isso vale mais que qualquer teste unitário.
2. **Isolamento entre motores.** Cada vertical é um módulo próprio, com domínio, regras e engine próprios. Um motor nunca importa o outro e nunca é alterado por causa de outro.

---

2. Contexto de negócio e regulatório

2.1 Vocabulário mínimo

- **Associação**: entidade que reúne os associados e administra os eventos.
- **Administradora (Mutualiza)**: sociedade anônima de objeto exclusivo, autorizada pela SUSEP, que passa a gerir os grupos de proteção mutualista.
- **SGA**: sistema de gestão da associação. Nos clientes atuais é o **Hinova**. O Mut não substitui o SGA hoje: ele lê exportações em xlsx do SGA e calcula o rateio por fora.
- **Evento**: sinistro. Tem protocolo, motivo, valor de reparo, participação do associado e custo.
- **Participação** (ou franquia): parcela que o associado paga do evento. `custo = reparo - participação`.
- **Cota**: unidade de peso do veículo no rateio. Carro mais caro tem mais cotas, e paga proporcionalmente mais.
- **Rateio**: divisão do custo do período entre as cotas elegíveis.
- **Integral**: evento de indenização total (V.F.I, furto, roubo).
- **Salvado**: o que sobra do veículo indenizado e é vendido. Reduz o custo (dedução).

- **Foto**: exportação do SGA tirada num momento. O fechamento trabalha sobre uma foto estática.

2.2 Regulatório (LC 213/2025 e Res. CNSP 491/2026)

O que importa para o código:

- A cobrança precisa **discriminar** taxa de administração, contribuição de rateio e contribuição de estabilização (art. 70 da resolução). Por isso o boleto da AVAP já nasce com esse split, mesmo com componentes zerados.
- A contribuição de estabilização é obrigatória (art. 69). A AVAP não tem uma hoje. Está em aberto (decisão D5, seção 8.15).
- O montante a ratear admite "outras receitas e despesas do grupo previstas no contrato de participação". É a base legal das linhas de soma e dedução da AVAP.
- A SUSEP pode pedir auditoria operacional a qualquer tempo. O fechamento lacrado, com trilha imutável e conciliação a zero, é a resposta técnica a isso.
- Vedação de partes relacionadas: a associação gerida não pode ter participação na administradora. Há uma proposta comercial de a associação ser cotista da administradora que pode esbarrar nisso. É risco jurídico, não técnico, mas vale saber que existe.

Os parâmetros detalhados (capital mínimo, prazos, sanções) estão resumidos em `docs/Roadmap de Desenvolvimento de Software_ Plataforma para Administradoras (LC 213_25).md` e no manual citado na memória do projeto. Antes de usar qualquer número regulatório em material formal, confira no texto oficial.

2.3 Pessoas e papéis

Este documento não usa nomes. Os papéis que você vai encontrar:

- **Responsável pelo projeto**: dono do produto, decide escopo e prioridade, e é quem fala com a Mutualiza.
- **Contato operacional da Mutualiza**: quem manda os arquivos do SGA e responde as dúvidas de regra de cada associação. É por esse contato que passam as perguntas abertas da AVAP.
- **Operador da associação**: quem faz o fechamento hoje, na mão.
- **Gestores da Mutualiza**: decisores da administradora (negociação comercial e societária).

Peça ao responsável pelo projeto os contatos e o acesso aos dados reais.

---

3. Mapa do repositório

Raiz: `/root/projetos/Mut`. Repositório privado no GitHub (`DuduOliveira-ia/Mut`). Branch de trabalho atual: `feat/admissao-verificada-sdd`. Branch principal: `main`.

```
Mut/
├── backend/                  API FastAPI + os três motores (v1)
│   ├── app/
│   │   ├── core/             motor ELLO (domain, rules, engine, ingestao_xlsx)
│   │   ├── autotruck/        motor AutoTruck (domain, rules, engine, fechamento,
ingestao...)
│   │   ├── avap/             motor AVAP (domain, rules, ingestao, selecao, engine,
historico, reconciliacao)
│   │   ├── api/              rotas: rateio.py, consultas.py (ELLO), autotruck.py,
avap.py
│   │   ├── schemas/          modelos pydantic de ELLO e AutoTruck
│   │   ├── repositories/     json_repo e supabase_repo (só ELLO)
│   │   ├── services/         rateio_service (ELLO)
│   │   ├── config.py, deps.py, main.py
│   ├── tools/                CLIs: reconciliar, reconciliar_avap, fechar_mes,
poc_remonta, geradores
│   └── tests/                113 testes
├── frontend/                 React 19 + Vite + Tailwind 4
│   └── src/                  App.jsx (ELLO + seletor Root), AutoTruck.jsx, AVAP.jsx
├── admissao-verificada/      MVP de onboarding em SDD (specs, ADRs, app Next.js,
harness)
├── docs/                     PRDs, relatórios, análises, decks, evidências
├── demos/                    kits de demonstração (ELLO, AutoTruck) e guias de acesso
├── simulacoes/               planilhas geradas (simuladores, homologação, PoC)
├── inputs/ e outputs/        entradas e saídas do v0 (scripts CLI da raiz)
├── decks/, mira-templates/,
│   mira.config.json, .mira/  toolkit Mira de slides (ver CLAUDE.md da raiz)
├── AVAP/, ELLO/, Remonta/    DADOS REAIS DE CLIENTE, no .gitignore. Não existem num
clone novo.
├── rodar_motor_rateio.py,
│   gerar_dados_simulados.py,
│   criar_webapp*.py          v0: pipeline em scripts que antecedeu a API
├── dados_simulados.json,
│   dados_autotruck.json      datasets sintéticos
├── PRD.md, CONTEXTO.md,
│   SETUP.md                  documentos da fase inicial (ELLO v0)
└── CLAUDE.md                 instruções do toolkit Mira
```

Observações importantes:

- **`AVAP/`, `ELLO/` e `Remonta/` não estão no git** (dados de cliente). Num clone novo essas pastas não existem. Peça os arquivos ao responsável pelo projeto e coloque-os nas mesmas pastas. Nunca commite arquivo real de cliente nem coloque dado real em teste versionado.
- **Dois mundos de código**: o v0 (scripts da raiz que geram xlsx e um HTML de demonstração) e o v1 (API + front). O v0 é histórico, mas `rodar_motor_rateio.py` continua sendo a referência de leitura da regra ELLO original.
- **`decks/`, `mira-templates/` e o `CLAUDE.md` da raiz** pertencem ao toolkit Mira, que gera apresentações HTML. Não têm relação com o motor. Se você for mexer em slides, leia o `CLAUDE.md`.

---

4. Colocando o ambiente de pé

4.1 Pré-requisitos

- Python 3.11 ou superior.
- Node 22 e npm.
- Os dados reais, se for trabalhar com AVAP ou ELLO (pastas ignoradas pelo git).

4.2 Backend

```
cd backend
python3 -m venv .venv
.venv/bin/pip install -e ".[dev]"
cp .env.example .env
```

Ajuste o `.env`. O exemplo e o default de `config.py` apontam para o caminho de outra máquina (`/home/ronaldim/Mut/dados_simulados.json`), então corrija:

```
RATEIO_REPOSITORY=json
RATEIO_JSON_PATH=/caminho/do/seu/clone/Mut/dados_simulados.json
API_CORS_ORIGINS=http://localhost:5174
```

Subir a API:

```
.venv/bin/uvicorn app.main:app --host 0.0.0.0 --port 8000 --reload
```

Documentação interativa em `http://localhost:8000/docs`. Saúde em `GET /health`.

4.3 Frontend

```
cd frontend
npm install
npm run dev        # Vite em 0.0.0.0:5174
```

O Vite faz proxy de `/rateio`, `/boletos`, `/veiculos`, `/autotruck`, `/avap` e `/health` para `localhost:8000`. A tela tem um seletor de produto no topo (ELLO, AutoTruck, AVAP).

4.4 Testes

```
cd backend
.venv/bin/python -m pytest -q
```

Use o Python do venv. O Python do sistema não tem pytest. Estado em 12/10/2026: **113 testes passando** (cerca de 1 minuto), com os dados reais da AVAP presentes.

Atenção: os testes que usam `AVAP/` fazem `skip` quando a pasta não existe. Num clone sem os dados, a suíte passa, mas muita coisa relevante fica pulada (a leitura dos arquivos reais,

o aceite de setembro, a API de demonstração). Passou sem os dados não é o mesmo que passou com os dados.

Lint: `ruff` está configurado (linha de 110 colunas). Os arquivos da AVAP estão limpos.

4.5 Mês de referência e dados de teste

Vertical	Dataset	Mês de referência
ELLO	`dados_simulados.json`	`05/2025`
AutoTruck	`dados_autotruck.json` (regenerável)	`12/2025`
AVAP	arquivos reais em `AVAP/`	`09/2026`

Sanidade do ELLO: `POST /rateio/calcular {"mes_ano_ref":"05/2025"}` deve devolver `valor_alvo_total = 716964.0` e `diferenca_alvo_vs_alocado = 0.0`.

4.6 Armadilhas operacionais que já custaram tempo

1. **Estado em memória.** Os repositórios json (ELLO, AutoTruck) e a sessão da AVAP vivem na memória do processo. Reiniciou o backend, zerou. As consultas `GET` só respondem depois do `POST` de cálculo.
2. **Processo preso em porta.** Pode haver uvicorn ou vite antigo ocupando 8000 ou 5174. Mate por PID.
3. **Nunca `pkill -f "uvicorn app.main:app"`.** O padrão casa com a própria linha do shell e derruba a sessão. Guarde o PID em arquivo e mate por ele.
4. **Sessão que cai mata processo em segundo plano.** Suba com `setsid nohup ... < /dev/null &`.
5. **Rotas catch-all.** Em ELLO e AutoTruck há rotas `/{mes:path}`. A ordem de registro importa: as específicas vêm antes. Já houve bug por causa disso (comentário em `main.py`).
6. **Formato do mês.** No corpo das requisições é `MM/AAAA`. Na URL da AVAP é `MM-AAAA` (a barra quebraria a rota). A função `_chave` normaliza.

---

5. Arquitetura e princípios

5.1 Camadas

```
SGA (xlsx)  ->  ingestao  ->  domain (dataclasses puras)  ->  engine (função pura)  ->
resultado
                                    ^
|
                                rules (parâmetros)                    api (FastAPI)
->  frontend (React)
```

- **domain**: dataclasses sem I/O. Imutáveis (`frozen=True`) onde possível.

- **rules**: parâmetros configuráveis (valores de degrau, limites de alerta, critérios). Mudar regra é mudar `rules`, não o engine.
- **ingestao**: leitores de xlsx. Falham alto quando o arquivo não confere.
- **engine**: funções puras, sem I/O, sem relógio, sem rede. Mesma entrada, mesma saída.
- **api**: orquestra, valida, traduz erro de domínio em HTTP.

5.2 Princípios que o time assumiu

1. **Determinismo e prova.** Todo motor tem invariantes (soma alocada igual ao alvo, soma dos boletos igual ao numerador) e é testado contra dado real.
2. **Falhar alto na leitura.** Os leitores comparam o que leram com o rodapé do próprio SGA e abortam se divergir. O SGA é pouco confiável (exporta duplicatas com valores diferentes), então desconfiança é o padrão.
3. **Nada de decisão silenciosa.** Exclusão ou inclusão manual exige justificativa e entra no log. Diferença de arredondamento é reportada, nunca redistribuída em silêncio.
4. **Isolamento.** Os motores não se importam. As mudanças fora do módulo são só aditivas: um `include_router` em `main.py`, um proxy em `vite.config.js`, um botão em `App.jsx`.
5. **Dado de cliente fora do git.** E nada de dado pessoal desnecessário: o leitor de eventos da AVAP não carrega o nome do associado.
6. **Cota é lida, não recalculada** (AVAP). O peso vem do SGA.

5.3 Dívida técnica conhecida

- Há código copiado entre `autotruck` e `avap` (ex.: leitura de xlsx). É deliberado, para manter o isolamento. Extrair um módulo comum só depois do aceite das três verticais, em tarefa própria (decisão D6).
- A API não tem autenticação. É uma demonstração. Antes de qualquer exposição real, isso precisa mudar.
- Persistência real (Supabase) existe só para o ELLO, e mesmo ali o modo default é json.

---

6. Motor ELLO

Arquivos: `backend/app/core/{domain,rules,engine,ingestao_xlsx}.py`, rotas em `app/api/rateio.py` e `app/api/consultas.py`, serviço em `app/services/rateio_service.py`.

6.1 Modelo

Valor fixo por cota por categoria de risco. Os eventos do mês **preenchem** esse alvo, não o definem.

São quatro categorias e quatro "potinhos" cumulativos (`core/rules.py`):

Potinho	Valor por cota	Quem paga
p1_roubo_furto	R$ 14	todas as categorias

Potinho	Valor por cota	Quem paga
p2_moto_fip	R$ 4	MOTO_FIP, VENTURE_1, VENTURE_2
p3_venture_1	R$ 18	VENTURE_1, VENTURE_2
p4_venture_2	R$ 18	VENTURE_2

Valor final por cota: ROUBO_FURTO R$ 14, MOTO_FIP R$ 18, VENTURE_1 R$ 36, VENTURE_2 R$ 54. Os degraus são **cumulativos**, não substitutivos.

6.2 Algoritmo (`core/engine.calcular_rateio`)

1. Elegíveis: veículo ativo e não funcionário.
2. Soma de cotas por categoria.
3. Para cada degrau: base de cotas das categorias que pagam, vezes o valor do degrau. A soma dos degraus é o **alvo do mês**.
4. Aloca eventos em ordem `data de ocorrência ASC`, `saldo pendente DESC`, `id`, até zerar o alvo. Admite **rateio parcial** (fatia o último evento). O que não coube vira `SALDO_RESTANTE`; o que coube inteiro vira `LIQUIDADO`.
5. Gera boletos: rateio do veículo (cotas vezes valor da categoria) mais produtos fixos (taxa administrativa, assistência 24h, outros).
6. Consolida.

Regras de exceção:

- **Funcionário**: não entra na base de cotas e não gera boleto.
- **Plano anual**: entra como peso no alvo, mas não gera boleto mensal.
- **Inativo**: fora de tudo.

Invariante: soma do rateado nos eventos = alvo. `diferenca_alvo_vs_alocado` deve ser 0,00.

6.3 Endpoints

`POST /rateio/calcular`, `POST /rateio/replay`, `POST /rateio/rodar-mes`, `POST /rateio/inspecionar-arquivo`, `GET /rateio/{mes}`, `GET /rateio/{mes}/eventos`, `GET /boletos/{mes}`, `GET /veiculos/{id}/extrato/{mes}`. Não existe `/rateio/auditoria`: a auditoria vem dentro do `consolidado`.

6.4 Situação com dados reais

Chegou o primeiro arquivo real da ELLO (30/09/2026). Só tem eventos: sem base de veículos, sem serviços fixos, sem data de ocorrência. Não dá para rodar o motor ainda. Há 100 linhas duplicadas, valores negativos e diferença entre o cabeçalho (R$ 4,75 mi) e os protocolos únicos (R$ 3,80 mi). Análise completa em `docs/RELATORIO_ADERENCIA_ELLO_dados_reais.md`.

Decisão tomada: a ELLO continua em valor fixo por cota. O modelo orientado a custo é do AutoTruck e não se mistura.

---

7. Motor AutoTruck

Arquivos: `backend/app/autotruck/*`, rotas em `app/api/autotruck.py`, front em `frontend/src/AutoTruck.jsx`. PRD em `docs/PRD_AutoTruck.md`.

7.1 Modelo

Repasse de custo. O custo do mês é dividido pelas cotas, **por segmento**: LEVES, MOTOS e PESADOS. LEVES e MOTOS são rateio variável com bases separadas. PESADOS (casco de caminhão) é um pote fixo. Terceiros também têm pote próprio.

Fórmula do custo variável: `(integrais + parciais - salvados - franquias) / cotas`. Recuperação de terceiros **não abate**.

7.2 Regras relevantes

- **Cota por FIPE**: até R$ 10 mil vale 1; cada faixa de R$ 10 mil soma 0,5; acima de R$ 450 mil vale cota especial (multiplicador 20).
- **Regra dos 75%**: se o reparo passa de 75% do valor, a perda parcial vira integral.
- **FIFO**: o evento postergado em mês anterior tem prioridade absoluta; depois data, tipo e valor.
- **Salvado** só abate quando liquidado financeiramente.
- **Denominador purista**: veículo ativo e adimplente.
- **Estabilização**: faixa-alvo de R$ 30 a 35 por cota, com ajuste por média trimestral acima de 5%. Existe um assistente de ajuste (`/autotruck/sugerir-ajuste` e `/aplicar-ajuste`).
- **Alerta de volatilidade**: um evento acima de 25% do pote.

7.3 Fechamento lacrado (o padrão que a AVAP vai copiar)

`autotruck/fechamento.py`: a "foto de fechamento" calcula SHA-256 dos insumos (veículos, eventos, parâmetros). Máquina de estados `ABERTO -> HOMOLOGADO -> EMITIDO`. A verificação de integridade detecta qualquer alteração posterior na fonte. Trilha imutável de auditoria. CLI `tools/fechar_mes.py` roda o fluxo D+0 a D+5.

7.4 Aceite

CLI de reconciliação: `cd backend && .venv/bin/python -m tools.reconciliar <FECHAMENTO.xlsx>`. Lê o FECHAMENTO real da associação (284 eventos) e reproduz a aba ANALITICO com **Δ = R$ 0,00 em 13 de 13 linhas**. Código de saída 0 é aprovado. Fixture em `backend/tests/fixtures/FECHAMENTO_referencia.xlsx`.

Armadilha: o AutoTruck tem **dois leitores de RATEIO** para dois layouts diferentes (`ingestao_partes.py` para o upload da tela, `ingestao.py` para a reconciliação). O FECHAMENTO consolidado só é lido pela CLI, não pela tela.

7.5 Outros artefatos

PoC Remonta (`tools/poc_remonta.py` , aprovada pela associação), simuladores xlsx com fórmulas nativas em `simulacoes/` , carta de homologação em `docs/HOMOLOGACAO_carta.md` , dicionário de dados e especificação de insumos em `docs/` .

---

8. Motor AVAP (foco deste guia)

Esta é a seção mais importante. Leia com calma.

Arquivos:

Arquivo	Responsabilidade
`backend/app/avap/domain.py`	dataclasses puras
`backend/app/avap/rules.py`	parâmetros (`RegrasAVAP`)
`backend/app/avap/ingestao.py`	leitores dos exports do SGA, parser de cota, conferência de rodapé
`backend/app/avap/historico.py`	leitor da planilha de rateio real (aceite) e classificação das linhas de soma e dedução
`backend/app/avap/selecao.py`	candidatos, alertas, decisões, duplicidade
`backend/app/avap/engine.py`	numerador, denominador, valor da cota, boletos, invariantes
`backend/app/avap/reconciliacao.py`	confronto do fechamento real contra o motor
`backend/tools/reconciliar_avap.py`	CLI de aceite
`backend/app/api/avap.py`	rotas `/avap/*` e sessão em memória
`frontend/src/AVAP.jsx`	tela (aba "AVAP")
`docs/REGRAS_RATEIO_AVAP.md`	**fonte única das regras**: cada regra com status (decidido, aberto que bloqueia, aberto que não bloqueia) e fonte
`docs/PRD_AVAP.md`	especificação técnica (parte descreve o alvo, ver 8.16)

8.1 O que é a AVAP e como ela faz o rateio

A AVAP é uma associação de proteção de **automóveis** (só `AUTOMOVEL` na base atual). Vem pela Mutualiza como terceira vertical. Hoje o fechamento é feito por um operador, com apoio de planilha, a partir de exportações do SGA Hinova.

Características do modelo (confirmadas em reunião e nos dados):

- **Repasse de custo.** O que se rateia é o custo dos eventos escolhidos, não um valor fixo por cota. Em todo evento rateado, `Valor Rateado = Valor Reparo - Valor Participação` , sem exceção (150 eventos em agosto, 103 em setembro).

- **Pote único.** Sem categoria, sem classificação de risco, sem segmento. Só cota.
- **Cota = 1 por R$ 5 mil de FIPE.** Carro de R$ 20 mil tem 4 cotas, de R$ 100 mil tem 20. Mas o motor **não calcula**: lê o peso que o SGA informa.
- **Sem rateio parcial e sem postergação.** Só entra evento totalmente finalizado, e entra inteiro.
- **Foto estática.** O rateio é uma foto do dia. Não entra nota fiscal nova durante o processo. O corte é à meia-noite do dia anterior ao fechamento.
- **Integrais são rateadas inteiras**, mesmo quando o pagamento ao associado é parcelado.
- **Terceiros não são rateados.** São vendidos como produto.
- **Linhas de soma e dedução** entram no numerador, além dos eventos.

8.2 A fórmula

```
                 Σ custo dos eventos selecionados
               + Σ linhas de SOMA
               - Σ linhas de DEDUÇÃO
valor_cota  =  ------------------------------------
                 Σ peso das cotas elegíveis

rateio_do_veículo = peso_da_cota x valor_cota
```

Implementação em `engine.calcular_rateio`:

- `custo` = soma de `custo_evento` dos eventos selecionados, arredondada a 2 casas.
- `somas` e `deducoes` = soma dos lançamentos por tipo (valores sempre positivos; o sinal vem do tipo).
- `numerador = custo + somas - deducoes`.
- `valor_cota = numerador / total_cotas`, **em precisão total**. Cada parcela do boleto é arredondada a 2 casas.
- `diferenca_arredondamento` = soma dos boletos menos numerador. É **reportada**, nunca redistribuída.

Invariantes verificadas (`ResultadoAVAP.invariantes`):

1. `arredondamento`: `|diferença| < 0,01 x número de boletos`.
2. `numerador_positivo`.
3. `protocolos_unicos`: nenhum protocolo duas vezes no mesmo fechamento.

Se não houver nenhum veículo elegível, o engine levanta `ValueError("Denominador zerado")`.

8.3 Entradas: os exports do SGA

São três tipos de arquivo. Todos xlsx, cabeçalho na linha 1, rodapé a partir de uma linha "Resumo".

a) Base de veículos (`Cotas_ativasAVAP_09_26.xlsx`, 30/09/2026). Colunas obrigatórias: Placa, Cota, Categoria, Tipo Veículo, Valor FIPE Veiculo, Situação Veículo, Situação Associado, Matrícula Associado.

Perfil real: **39.242 veículos e 361.530 cotas** (confere com o rodapé). Só **22.539 veículos ativos, com 231.628 cotas**; 16.703 inadimplentes (43% dos veículos, apesar do nome "cotas ativas"). Placa pode vir vazia (69 casos), FIPE pode ser 0 ou vazia (80 casos). Depois das exclusões do motor (peso 0, associado pendente, sem placa e sem FIPE), o denominador de setembro fica em **231.556 cotas**.

b) Eventos, dois layouts (`foto28-09.xlsx` e `foto30-09.xlsx`). O leitor detecta o layout pela presença da coluna `Placa` (layout "28/09") ou `Tipo Veiculo` (layout "30/09"). O layout de 30/09 repete um bloco de colunas vazias depois de Tipo Evento; o leitor usa sempre a **primeira** ocorrência de cada nome de coluna. O layout recomendado é o de 28/09 com `Tipo Veiculo` acrescentado.

Colunas obrigatórias: Protocolo, Situacao, Data Finalizacao, Valor Reparo, Valor Participacao, Valor CustoEvento, Previsao Valor Reparo, Motivo, Valor Rateado, Data Cadastro Evento, Cota, Categoria, Modelo, Tipo Evento.

Perfil real: 3.104 eventos de 01/2025 a 30/09/2026, somando R$ 39.122.991,79 de custo (bate com o rodapé).

c) Planilha de rateio real (`rateio_agosto.xlsx`, abas `Rateio_agosto` e `Rateio_setembro`). Não é export do SGA: é o fechamento que a associação montou. Cabeçalho exato: `Tipo Veiculo, Placa, Protocolo, Descrição, Data, Valor Reparo, Valor Participação, Valor Rateado`. Linhas **com placa** são eventos. Linhas **sem placa** são lançamentos de soma ou dedução. Serve **só para conciliação**: não alimenta o cálculo.

8.4 Conferência de rodapé (falhar alto)

Os dois leitores do SGA comparam o que leram com o rodapé e levantam `ConferenciaRodapeError` se divergir (a menos que se passe `estrito=False`).

- Veículos: contagem e soma de cotas.
- Eventos: contagem, reparo, participação e custo.

O rodapé de eventos traz "Valor a Ratear" e "Valor Rateado" **duas vezes** (eventos e eventos relacionados). A segunda ocorrência vira a chave `_relacionados`.

Números do SGA vêm como texto em vários formatos (`"25200.00"`, `"361,530.00"` com vírgula de milhar). A função `_num` normaliza. **Cuidado**: o formato é o do export (ponto decimal, vírgula de milhar), não o brasileiro.

Rótulo de layout ou coluna ausente levanta `LayoutDesconhecidoError`.

8.5 Parser de cota

`ingestao.parse_cota(rotulo) -> (peso, sufixo)`.

O peso é o número entre parênteses **no fim** do rótulo. Exemplo: `COTA 6,0 - (25.000,01 A 30.000,00) - VP (6.00)` tem peso 6,0 e sufixo `VP`.

Regras:

- Regex de peso: `\(\s*(\d+(?:[.,]\d+)?)\s*\)\s*$`. Tolera vírgula ou ponto.
- Sufixo: `VP`, `DP`, `CP`, `TP` ou nenhum. Guardado em `sufixo_cota`. **O significado é desconhecido** (decisão D3), por isso não muda o peso.
- `COTA 0,0 - PRODUTOS AVULSOS (0.00)` tem peso 0.
- **Nunca recalcular pela FIPE.** Em 2.270 veículos (6%) o peso do SGA difere de `ceil(FIPE / 5000)`, por ajuste legítimo ou por falta de reenquadramento. O peso do SGA é a verdade.
- Rótulo sem peso no fim levanta `ValueError`.
- Os rótulos reais têm variações de digitação (`20.0000,00`, `R$ 210.000,00`, `15.000.01`, `COTA 80.00 -`, `COTA 8,0 (35.000,01...` sem hífen). Como o peso é lido do fim, a maioria é irrelevante. O sufixo só é capturado quando segue o padrão `- XX (n)`.

8.6 O campo `Valor Rateado` do SGA e a lição mais cara do projeto

O leitor classifica o campo em quatro estados (`EstadoRateio`):

Valor na planilha	Estado	O que o SGA quer dizer
igual ao custo	`RATEADO`	já entrou num fechamento
`"0.00"`	`NA_FILA`	está na fila do fechamento em curso
vazio	`NAO_RATEADO`	fora da fila
qualquer outro	`RATEADO_DIVERGENTE`	alerta (a AVAP não faz rateio parcial)

Este campo NÃO define o fechamento real. É só um atributo lido. Isso foi descoberto em duas etapas e rendeu retrabalho:

1. Primeiro, "vazio" foi lido como "fora do rateio". Errado: segundo a reunião, vazio significa ainda não rateado.
2. Depois, as fotos de 28/09 e 30/09 foram lidas como "73 rateados + 7 integrais retirados". Também errado. Quando chegou a planilha de rateio real (10/10/2026), viu-se que **6 dos 7 "retirados" estavam dentro** do rateio de setembro. Os integrais ficam com o campo vazio no SGA e mesmo assim são rateados.

Consequência de projeto, que está no código: **só `NA_FILA` é candidato por regra**; o evento com campo vazio entra apenas por decisão manual `INCLUIR` do operador, com justificativa. Isso reproduz o fechamento real de setembro:

- A fila de 28/09 tinha 80 eventos (R$ 849.115,48, igual ao "Valor a Ratear" do rodapé).
- 79 deles entraram. O 80º (um furto de R$ 67.092,44) foi excluído. O motivo da exclusão ainda é pergunta aberta à associação.
- 24 eventos com campo vazio foram incluídos à mão (22 integrais, 1 colisão, 1 continuação de processo).
- 79 + 24 = **103 eventos, R$ 1.733.393,65**, igual à planilha.

Ou seja, a seleção real é: fila do SGA, mais decisões do operador. O motor modela exatamente isso. O teste `test_real_selecao_setembro_reproduz_fechamento` trava esse comportamento.

8.7 Seleção dos eventos (`selecao.selecionar`)

Assinatura: `selecionar(eventos, regras, decisoes, data_fechamento, ja_rateados, datas_anteriores) -> ResultadoSelecao`.

Candidato por regra (todas as condições):

1. `situacao == "FINALIZADO"`.
2. `tipo_evento == "Evento Associado"`.
3. `estado_rateio` em `regras.estados_candidatos` (default: só `NA_FILA`).
4. `data_finalizacao < data_fechamento` (se informada). É o corte à meia-noite do dia anterior.
5. Protocolo não consta em `ja_rateados`.

Ficam fora sem precisar de decisão: finalizado sem rateio, cancelado, pendente de nota fiscal, veículo entregue, terceiros, já rateados.

Duplicidade (chave: protocolo):

- Duplicata **idêntica**: deduplica e registra na auditoria.
- Duplicata **com valores diferentes**: **bloqueia** a homologação até haver decisão. No SGA isso pode ser recadastro com outro valor ou outra nota.

Alertas (o motor não exclui sozinho, pede decisão):

Código	Condição	Parâmetro
A1	protocolo com menos de 10 dígitos ou prefixo de ano abaixo do mínimo	`ano_minimo_protocolo = 2023`
A2	`custo_evento <= 0`	
A3	participação zero e custo abaixo de R$ 500	`custo_minimo_sem_participacao`
A4	evento sozinho acima de 10% do numerador-base	`pct_concentracao`
A5	motivo integral (V.F.I, FURTO, ROUBO)	`motivos_integrais`
A6	data de finalização mudou em relação à foto anterior	recebe `datas_anteriores`
A7	`custo != reparo - participação`	tolerância R$ 0,01

Sem decisão para um evento com alerta, ele vira **pendência** e o fechamento **não é homologável**.

Decisões do operador (`DecisaoEvento`): `INCLUIR` ou `EXCLUIR`, sempre com justificativa não vazia. Duas decisões para o mesmo protocolo levantam erro. Decisão para protocolo que não existe na foto vira **bloqueio**. `INCLUIR` de protocolo já rateado em outra competência também bloqueia.

Resultado (`ResultadoSelecao`): `selecionados` (com origem `REGRA` ou `DECISAO` e os alertas), `nao_selecionados` (com motivo legível), `alertas`, `pendencias`, `bloqueios`, `auditoria`. A propriedade `homologavel` é `not pendencias and not bloqueios`.

Detalhe de implementação: o alerta A4 usa como base a soma dos eventos "em jogo" (os que passam pela regra, ou foram incluídos), não o numerador final com linhas de soma e dedução.

8.8 Lançamentos: linhas de soma e dedução

Linhas que entram no numerador além dos eventos. O valor é sempre positivo; o sinal vem do tipo.

Catálogo observado em agosto e setembro (classificação em `RegrasAVAP.codigos_soma` e `codigos_deducao`, por trecho da descrição em maiúsculas):

Tipo	Código	Descrição típica	Observação
SOMA	`E_SAUDE`	E-SAUDE	benefício; em agosto aparece 3 vezes (pergunta aberta)
SOMA	`RASTREAMENTO`	RASTREAMENTO	cerca de R$ 118 mil por mês; se é custo do pote ou repasse a terceiro está em aberto
SOMA	`INADIMPLENCIA`	INADIMPLÊNCIA	cobre o que os inadimplentes deixaram de pagar
DEDUCAO	`SALVADOS`	RECEITA COM VENDAS DE SALVADOS	salvado vendido no período
DEDUCAO	`RECUP_INADIMPLENCIA`	RECEITA INAD. / RECEBIDOS	recuperado no mês seguinte
(qualquer)	`NAO_CLASSIFICADO`	sem descrição reconhecida	cai aqui, e vira linha informativa na reconciliação

`historico.classificar_linha(descricao, valor)`: o **sinal** do valor define o tipo (negativo é dedução), a **descrição** define o código. Isso é uma heurística. A linha de setembro de -R$ 129.253,00 vem sem descrição e fica `NAO_CLASSIFICADO` (provavelmente salvados, como em agosto, mas é pergunta aberta).

Custos de guincho e pátio do salvado **não** são linha: já estão dentro do custo do evento. Deduções não têm efeito contábil ou tributário aqui: entram cheias.

8.9 Denominador e a decisão D2

Soma do `peso_cota` dos veículos **elegíveis**. `engine._motivo_fora` decide, na ordem:

1. `PESO_ZERO`: peso menor ou igual a zero.
2. `ASSOCIADO_PENDENTE`: situação do associado em `situacoes_pendentes` (pendente, pendente revistoria, pendente adesão).
3. `SEM_PLACA_SEM_FIPE`: sem placa e sem FIPE (D4).
4. Conforme o `criterio_elegibilidade`: - `VEICULO`: situação do veículo precisa ser ATIVO. - `ASSOCIADO`: situação do associado precisa ser ATIVO. - `AMBOS`: as duas. - `TODOS`: não filtra por situação.

O motor devolve também `veiculos_fora`, com a contagem por motivo.

D2 está decidido: só o veículo ativo entra. A associação respondeu que o que conta é o **veículo**, não o associado, e que veículo inadimplente fica fora do rateio, com a compensação vindo pelas linhas de inadimplência (soma) e de recuperados (dedução no mês seguinte). Fontes: levantamento 1.18 e a resposta registrada na análise de agosto e setembro, seção 2. Default em `rules.py` e na API: `VEICULO`.

Setembro/2026: 231.556 cotas, **R$ 7,78 por cota**.

Os outros critérios (`TODOS`, `ASSOCIADO`, `AMBOS`) continuam no código só para comparação, e a tela mostra os quatro lado a lado. Não trate isso como dúvida em aberto. Houve uma hipótese de divisão por todas as cotas (R$ 4,99, perto de R$ 5,00), mas ela foi descartada: além de contrariar a resposta da associação, com ela os inadimplentes (cerca de 129 mil cotas) deixariam de pagar uns R$ 650 mil por mês, enquanto a linha de inadimplência real fica entre R$ 100 e 110 mil. Com veículos ativos, essa linha equivale a umas 14 mil cotas, o que é coerente.

O boleto real da associação traz só o total da mensalidade, sem separar rateio, taxa e produtos. Por isso **o boleto não serve de aceite**; o aceite é a planilha de rateio.

8.10 Boleto e o split do art. 70

`BoletoAVAP` por veículo: `placa`, `matricula`, `peso_cota`, `contribuicao_rateio` (= `peso x valor_cota`), `contribuicao_estabilizacao`, `taxa_administracao`, `produtos_beneficios`. O `total` é a soma dos quatro componentes.

No MVP só `contribuicao_rateio` é diferente de zero. Os outros três são parâmetros por cota em `RegrasAVAP`, todos com default 0:

- `taxa_administracao_por_cota` (vai para a administradora);
- `produtos_beneficios_por_cota` (assistência 24h e afins, vai para a associação);
- `contribuicao_estabilizacao_por_cota` (vai para o grupo, aguarda D5).

O split existe desde o início para a estrutura não mudar quando esses valores forem definidos.

8.11 Reconciliação e aceite (`reconciliacao.py` e CLI)

O aceite da AVAP é **reproduzir o fechamento real com diferença zero**. A CLI:

```
cd backend
.venv/bin/python -m tools.reconciliar_avap AVAP/rateio_agosto.xlsx \
    --aba Rateio_setembro \
    --eventos AVAP/foto30-09.xlsx \
    --veiculos AVAP/Cotas_ativasAVAP_09_26.xlsx \
    --fila AVAP/foto28-09.xlsx \
    [--valor-cota 7.78]
```

Código de saída: 0 aprovado, 1 reprovado, 2 aba inexistente.

O que `reconciliar()` confere, em grupos:

1. **Eventos**: protocolos do rateio ausentes no SGA, e valores divergentes do SGA. Esperado: zero em ambos.
2. **Seleção** (só se vier `--fila`): monta decisões derivadas do histórico (INCLUIR o que está no rateio real, EXCLUIR o que estava na fila e ficou de fora) e confere quantidade e custo selecionados. Também exige zero pendências.
3. **Totais**: custo e quantidade de eventos.
4. **Linhas**: linhas sem classificação aparecem como `INFO`.
5. **Numerador e valor da cota por critério**: roda o motor para os quatro critérios de elegibilidade e compara com o total do histórico.

Status de cada linha: `OK`, `FALHA`, `INFO` (informativo) e `PENDENTE` (falta insumo). O resultado é aprovado quando não há nenhuma `FALHA`.

Sem `--valor-cota`, o valor por cota sai como `PENDENTE` (informativo) para cada critério, com a observação "critério vigente (D2)" na linha `VEICULO` e "só comparação" nas outras. Com `--valor-cota`, os critérios que não batem aparecem em `FALHA` por construção; leia só a linha do critério vigente.

Resultados atuais (com os dados reais):

Fechamento	Resultado	Total a ratear	Detalhe
Setembro/2026	**aprovado**	R$ 1.802.067,92	103 eventos, R$ 1.733.393,65
Agosto/2026	reprovado	R$ 1.806.046,24	150 eventos, R$ 1.775.640,36; 3 divergências conhecidas

Divergências de agosto (já perguntadas à associação): o protocolo 2026779124 não existe na foto de 30/09; os protocolos 2026782899 e 2026795888 têm valor de reparo diferente do SGA (o primeiro 11.092,69 na planilha contra 0 no SGA, o segundo 28.217,80 contra 35.426,80). Há hipótese de ajuste manual, a confirmar. O teste `test_cli_agosto_reprova_pelas_tres_divergencias_conhecidas` documenta isso.

Não confunda: "reprova agosto" aqui é o resultado **esperado e conhecido**, não um bug.

8.12 API (`/avap`)

O estado fica em `_SESSOES`, um dicionário em memória por competência. Reiniciar o backend apaga tudo. Qualquer mudança de seleção invalida o cálculo anterior (`resultado = None`).

Fluxo previsto: inspecionar arquivos, selecionar, decidir pendências, lançamentos, calcular, consultar.

Método e rota	Função
`POST /avap/inspecionar-arquivo`	valida um export (slot `VEICULOS` ou `EVENTOS`) com conferência de rodapé; devolve um resumo ou o erro. Não monta fechamento

Método e rota	Função
`POST /avap/selecionar`	recebe os dois xlsx e `mes_ano_ref` (e `data_fechamento` opcional), cria a sessão e devolve a seleção
`POST /avap/demo-setembro`	carrega os arquivos reais de 09/2026 do servidor e reproduz decisões e lançamentos do fechamento real. Fallback de estande
`GET /avap/{mes}/selecao`	seleção atual
`POST /avap/{mes}/decidir`	grava decisões (`protocolo` , `decisao` , `justificativa` com pelo menos 3 caracteres)
`POST /avap/{mes}/lancamentos`	substitui os lançamentos; cada item tem `descricao` e `valor` **com sinal** (negativo deduz)
`GET /avap/{mes}/lancamentos`	lista os lançamentos
`POST /avap/{mes}/calcular`	calcula com o critério informado; devolve o resultado e o comparativo dos quatro critérios
`GET /avap/{mes}/resultado`	último resultado
`GET /avap/{mes}/eventos`	eventos incluídos ou não incluídos, paginado
`GET /avap/{mes}/boletos`	boletos paginados, com filtro `q` por placa ou matrícula
`GET /avap/{mes}/boletos.csv`	exporta o CSV

Regras de erro:

- `404` se não há sessão para a competência.
- `409` ao calcular com pendências ou bloqueios ("Fechamento não homologável"), sem eventos selecionados ou sem veículos elegíveis; e ao pedir resultado antes de calcular.
- `400` para critério inválido.
- `422` para arquivo inválido (rodapé não confere, layout desconhecido).

Ponto de atenção: `/avap/inspecionar-arquivo` captura qualquer `Exception` de leitura e a devolve como badge de erro, com `# noqa: BLE001` justificado. É intencional.

O CSV usa `;` como separador e vírgula decimal.

8.13 Tela (`AVAP.jsx`)

Abas: Rodar Mês, Seleção, Lançamentos, Resultado, Boletos. O botão "Demonstração 09/2026" chama `/avap/demo-setembro` (leva uns 5 segundos, porque lê 39 mil veículos). A tela usa o critério vigente (`VEICULO`) e mostra os outros três ao lado, só como comparação. Placas aparecem mascaradas por padrão (três primeiros caracteres), pensando em demonstração pública.

Contexto: a tela foi feita para a Fenaprove (18 e 19/11/2026, Expominas, Belo Horizonte), onde o objetivo é mostrar o rateio ao vivo.

8.14 Testes da AVAP

Quatro arquivos, todos em `backend/tests/`:

- `test_avap_ingestao.py` (22 testes): parser de cota, estados do campo, leitura dos dois layouts, rodapé aprovado e reprovado, duplicata, layout desconhecido, e os arquivos reais.
- `test_avap_fase1.py`: classificação de linhas, regra de candidato, cada alerta, decisões, duplicata, engine (numerador, boletos, D2, D4, split, denominador zerado, arredondamento) e os aceites reais de seleção e total de setembro.
- `test_avap_reconciliacao.py`: aprovação, reprovação, `--valor-cota` decidindo critério, CLI de setembro (aprova), CLI de agosto (reprova pelas três divergências), aba inexistente.
- `test_avap_api.py`: inspeção, bloqueio de cálculo com pendência e liberação após decidir, demo de setembro, critério inválido e competência inexistente.

Os testes "reais" usam `AVAP/` e são pulados se a pasta não existir. Os demais usam dados sintéticos criados no próprio teste.

Convenção útil dos testes: a função `P(n)` converte números pequenos em protocolos de 10 dígitos com prefixo 2026, para não disparar o alerta A1 sem querer.

8.15 Decisões abertas (D1 a D8)

#	Decisão	Estado	O que falta
D1	Integrais entram?	**Resolvida**: são rateadas inteiras	critério de corte: 39 integrais (R$ 1,88 mi) ficaram fora de agosto e setembro; lista em `docs/AVAP_integrais_fora_do_rateio_ago_set_2026.md`
D2	Elegibilidade do denominador	**Resolvida**: só veículo ativo (`VEICULO`)	nada
D3	Sufixos VP, DP, CP, TP	aberta	resposta da associação; hoje não alteram o peso
D4	Peso 0, FIPE 0 ou sem placa	assumido: ficam fora, com alerta	confirmação
D5	Estabilização	assumido: 0 no boleto	definição da administradora e do atuário
D6	Reuso de código entre verticais	decidido: cópia adaptada	extrair módulo comum só após o aceite das três
D7	Catálogo de linhas de soma e dedução	parcialmente observado	fórmula da inadimplência, E-SAUDE x3, linha sem descrição de -R$ 129.253

#	Decisão	Estado	O que falta
D8	Layout do arquivo de cobrança para o SGA	assumido: CSV por placa e matrícula	operação da associação

A lista do que ainda se pergunta à associação está em `docs/REGRAS_RATEIO_AVAP.md`, seção 8. Hoje **só os 3 protocolos de agosto bloqueiam** o aceite. O resto (corte dos integrais, E-SAUDE, linha sem descrição, fórmula da inadimplência, rastreamento, sufixos) não bloqueia. O furto 2026819916 está com o presidente da associação. **Nunca pergunte algo marcado como decidido nesse documento**: já aconteceu, e queima a credibilidade do projeto com o cliente.

8.16 O que o PRD diz e o código ainda não tem

O `docs/PRD_AVAP.md` é a especificação, mas parte dele descreve o **alvo**, não o que existe. Antes de confiar numa seção, confira contra o código. Divergências conhecidas:

1. **Módulos previstos e inexistentes.** O PRD (seção 14) lista `fechamento.py` (foto com lacre SHA-256 e máquina de estados), `service.py`, `repo.py`, `config.py` e `gerar_dados.py` (dataset sintético) em `app/avap/`. **Nenhum existe.** Também não existe `app/schemas/avap.py`: os schemas pydantic estão dentro de `api/avap.py`.
2. **Endpoints diferentes.** O PRD prevê `/avap/importar`, `/{competencia}/simular`, `/homologar`, `/emitir`, `/auditoria`. A API real tem `/selecionar`, `/decidir`, `/lancamentos`, `/calcular` e outras (seção 8.12). Não há homologar, emitir nem auditoria persistida.
3. **Sem lacre nem estados.** Não há hash dos insumos nem transição ABERTO, HOMOLOGADO, EMITIDO na AVAP. Isso é a fase 3.
4. **Sem persistência.** A sessão é em memória.
5. **Sem dataset sintético da AVAP.** O PRD promete um gerador calibrado versionado; hoje os testes reais dependem de `AVAP/` e os demais montam dados mínimos à mão.
6. **Aceite descrito de forma antiga.** O relatório inicial trata o campo `Valor Rateado` como verdade. Ele tem no topo o aviso de documento histórico.
7. **Nome da CLI.** O PRD diz `python -m tools.reconciliar_avap <histórico.xlsx>`; a real exige `--aba`, `--eventos` e `--veiculos`.
8. **Docs antigos com premissas superadas.** Relatório, levantamento, análise de agosto e setembro e as listas de integrais e de retirados descrevem entendimentos que mudaram. Todos têm no topo o aviso de documento histórico. A referência é `docs/REGRAS_RATEIO_AVAP.md`, junto com o código.

8.17 Fases e próximos passos

Fase	Entrega	Estado
0	`ingestao.py`: parser de cota e conferência de rodapé	feita, commit `d4629f6`

Fase	Entrega	Estado
1	seleção, engine, lançamentos, histórico; aceite da seleção	feita, commit `3372584`
2	CLI `reconciliar_avap` e aceite principal	feita, commit `a980c2b` (setembro aprova; agosto depende só dos 3 protocolos)
3	fechamento lacrado, API e tela	**parcial**: API e tela de demonstração commitadas (`317e67e`); lacre, estados e persistência não
4	estabilização e split completo do boleto	não iniciada (D5)

Com o D2 decidido, a fase 3 (lacre, estados, persistência) pode andar. Para a feira, a prioridade continua sendo a tela de demonstração funcionando de forma robusta.

8.18 Armadilhas e lições da AVAP

1. **Não use `Valor Rateado` do SGA para decidir o que entra.** Só como atributo. (Seção 8.6.)
2. **Competência não é data de finalização.** A competência do evento é a do fechamento em que ele foi rateado. O fechamento de setembro levou eventos finalizados em agosto, e o de agosto levou eventos finalizados em setembro (11 e 14 casos). A coluna `Data` da planilha de rateio é a data do evento (2025 e 2026), não a de finalização.
3. **Placa não é chave da vida do veículo.** Um veículo indenizado pode voltar com outro associado e outro ID mantendo a placa. Para o boleto, o par placa e matrícula é o que se usa.
4. **Associado com vários veículos.** Inadimplência é por **veículo**: o mesmo associado pode pagar alguns boletos e não outros.
5. **O SGA é pouco confiável.** Exporta protocolos duplicados com valores diferentes. Por isso a duplicidade divergente bloqueia a homologação.
6. **Protocolos com tamanhos diferentes.** 8, 9 ou 10 dígitos. O prefixo de 4 dígitos é o ano. Protocolos antigos (2012, 2014, 2022) existem nos dados reais e disparam A1.
7. **Dado pessoal.** O leitor de eventos não carrega o nome do associado. Mantenha assim. Placas ficam mascaradas na tela por padrão.
8. **Números em texto.** `Valor Participacao` e `Valor CustoEvento` vêm como texto. Use `_num`, não `float()` direto.
9. **Não reabra decisões.** Antes de propor pergunta ao cliente ou mudar uma regra, confira o status em `docs/REGRAS_RATEIO_AVAP.md`. O D2 já foi reaberto por engano uma vez, a partir de uma hipótese escrita depois da resposta da associação.
10. **Uma fonte só.** Ao chegar resposta nova ou mudar um entendimento, atualize o item em `docs/REGRAS_RATEIO_AVAP.md` (status e fonte). Não crie outro documento nem empilhe blocos de "CORREÇÃO" nos antigos: foi isso que espalhou as regras e gerou confusão.

8.19 Como fazer três tarefas comuns na AVAP

Adicionar um novo alerta de seleção. Em `selecao._alertas`, acrescente a condição e o código; se tiver parâmetro, coloque em `RegrasAVAP`; cubra com um caso no `test_avap_fase1.py` (veja a parametrização de `test_alertas_bloqueiam_ate_haver_decisao`); atualize a tabela de alertas do PRD.

Adicionar um novo código de lançamento. Acrescente a tupla `(trecho da descrição, CÓDIGO)` em `codigos_soma` ou `codigos_deducao` de `RegrasAVAP` e um caso na parametrização de `test_classificar_linha`. Lembre que a ordem da tupla importa: o primeiro trecho que casar vence.

Registrar uma resposta nova da associação. Atualize o item em `docs/REGRAS_RATEIO_AVAP.md` (status, fonte e data). Se a resposta muda o cálculo, ajuste `RegrasAVAP` (e `CRITERIO_PADRAO` em `api/avap.py`, se for o denominador), cubra com teste, rode a suíte e a CLI `reconciliar_avap` para agosto e setembro, e alinhe o PRD.

---

9. Admissão Verificada (MVP em SDD)

Pasta `admissao-verificada/`. É o começo da cadeia: hoje o motor de rateio é o fim de um processo cuja entrada é manual. Este MVP constrói a **admissão do associado** (cadastro e vistoria antifraude, inspirada no App Visto) e gera a base que produz os arquivos BASE e RATEIO que o motor aprovado já lê.

Critério de sucesso (binário): admitir N associados, lançar eventos do mês, exportar `BASE.xlsx` e `RATEIO.xlsx`, e o motor aprovado rodar sobre eles com `detectar_tipo` correto e invariante zero. **O motor não é alterado**: a integração é por arquivo.

Metodologia: **Spec Driven Development**. As especificações vêm antes do código:

- `PROJETO.md`, `specs/MVP-SCOPE.md`, `specs/SPEC-000` a `SPEC-008`, `specs/RASTREABILIDADE.md`, `specs/REVISAO-SDD.md`.
- ADRs: `ADR-001` (arquitetura), `ADR-002` (oito convenções canônicas ancoradas no motor), `ADR-003` (denominador é soma de cotas por segmento, não contagem de veículos).

Entregas:

- `reference/exportador.py` e `reference/harness.py`: exportador e harness de reconciliação em Python, espelhando o leitor real do motor. Rodam como teste na suíte do backend (`test_admissao_harness.py`).
- `reference/ts_parity/`: prova que o exportador em TypeScript gera arquivos aceitos pelo motor (`test_export_ts_parity.py`).
- `app/`: Next.js 15, Supabase e TypeScript. Nove migrations SQL com RLS por claim JWT, auditoria imutável por trigger, e telas/rotas de associados, veículos, vistorias, situações de boleto, eventos e exportação. Compila (`tsc` sem erros, `next build` com 24 rotas).

O que falta: aplicar as migrations num projeto Supabase real, registrar o hook `custom_jwt_claims`, criar o bucket `vistorias`, completar os endpoints listados nos resumos das specs (PATCH por id, importação CSV, refazer foto) e rodar a esteira de ponta a ponta. **Não foi executado contra um Supabase real.**

Há um PR aberto (#1) com essa frente. Esta frente é independente da AVAP, embora ambas partilhem a branch atual.

---

10. Material comercial e de estratégia

Está em `docs/` e **não é código**, mas dá o porquê de várias decisões:

- Decks de apresentação para a Mutualiza (`apresentacao_mutualiza_defensibilidade_v1.x.html` e `.pdf`). A tese atual é a **defensibilidade**: o motor entrega números auditáveis que protegem o diretor responsável numa auditoria da SUSEP. A abordagem anterior, de "plataforma de cinco módulos" contra o Hinova, foi aposentada e movida para `docs/arquivo/`.
- Valoração, mesa de sociedade e registro de entendimentos (`valoracao_rateio_aporte`, `guia_mesa_sociedade_particular`, `registro_entendimentos_plataforma_rateio`, `docs/bp_newco/`).
- Roadmap de plataforma para administradoras (LC 213/25) em quatro fases: fundação, core (onde o motor de rateio entra), compliance e multi-tenant.

Pontos para você levar em conta:

- O preço de mercado do rateio é da ordem de centavos por placa, recorrente. O motor é vendido como **melhor motor de rateio**, sem a promessa de substituir o Hinova.
- A parte comercial e societária está em negociação. Valores e percentuais não estão neste documento de propósito. Se precisar, peça ao responsável pelo projeto.
- Material formal segue as regras de estilo da seção 12.

---

11. Estado atual, pendências e próximos passos

Situação em 12/10/2026.

11.1 Git

- Branch: `feat/admissao-verificada-sdd`.
- Últimos commits da AVAP (todos no GitHub): `b6d50b5` (fonte única das regras e D2 decidido), `317e67e` (tela de demonstração), `a980c2b` (fase 2), `3372584` (fase 1), `d4629f6` (fase 0), `56699cb` (docs).
- **Não commitado** (fora da AVAP): `docs/RELATORIO_ADERENCIA_ELLO_dados_reais.md`, `docs/bp_newco/`.
- Regra de trabalho: **só commitar quando o responsável pelo projeto pedir.**

11.2 Prioridades

1. **Fenaprove (18 e 19/11/2026).** Deixar a tela AVAP robusta para demonstração ao vivo. Decidir se a demonstração pública usa dados reais ou uma base anonimizada (hoje as placas ficam mascaradas, mas a base é real).
2. **Aceite de agosto da AVAP.** Depende só da resposta sobre os 3 protocolos (2026779124, 2026782899, 2026795888).
3. **Fase 3 da AVAP** (lacre, estados, persistência). Já pode começar.
4. **Dúvidas que não bloqueiam** (corte dos integrais, linhas de soma e dedução): ver `docs/REGRAS_RATEIO_AVAP.md`, seção 8.
5. **ELLO com dados reais.** Falta a base de veículos, serviços fixos e um fechamento anterior para conciliar.
6. **Admissão Verificada.** Provisionar o Supabase e rodar a esteira.
7. **Política de versionamento de `.xlsx`.** Único entregável antigo do MVP original ainda em aberto.

11.3 Riscos a manter no radar

- Estado em memória: demonstração que reinicia o backend perde a sessão.
- API sem autenticação.
- Dados reais de cliente em máquina de desenvolvimento.
- Risco jurídico da associação cotista da administradora frente à vedação de partes relacionadas.
- Premissas que mudaram de uma semana para a outra (a AVAP é o exemplo). Sempre validar contra o fechamento real, não contra a leitura de um campo.

---

12. Convenções de trabalho

Idioma: pt-BR, com acentuação correta, em código visível ao usuário, docs e mensagens.

Estilo de texto (vale para qualquer documento, tela ou mensagem):

- sem travessão (use vírgula, dois-pontos ou parênteses);
- sem emojis e sem ícones decorativos;
- tom direto e profissional, sem informalidade e sem jargão desnecessário;
- sem nomes de pessoas em documentos formais (use o papel);

Código: Python 3.11, `ruff` com linha de 110 colunas, tipos nos domínios, dataclasses imutáveis, engine sem I/O. Comentários explicam o porquê e citam o item do PRD quando existir.

Testes: todo comportamento novo de regra ganha teste. Aceite sobre dado real usa `skipif` quando a pasta não existe, e nunca embute dado real no teste.

Git: mensagens no formato `feat(avap): ...`, `docs(...)`, `fix(...)`. Commit só sob pedido. Nunca commitar `.env`, `AVAP/`, `ELLO/`, `Remonta/`.

Documentação viva: as regras de negócio vivem num documento de referência por vertical (na AVAP, `docs/REGRAS_RATEIO_AVAP.md`). Quando uma premissa mudar, atualize esse documento com status, fonte e data. Os documentos antigos ficam como histórico, com aviso no topo.

Mudanças fora do módulo: ao tocar `main.py`, `App.jsx` ou `vite.config.js`, faça mudança estritamente aditiva, para não afetar as outras verticais. Rode a suíte completa.

---

13. Glossário

Termo	Significado
A1 a A7	alertas de seleção da AVAP
Alvo	valor total a ratear no mês (ELLO: soma dos potinhos)
Aceite	prova de que o motor reproduz o fechamento real com diferença zero
Administradora	sociedade que gere os grupos, exigida pela LC 213/2025
Candidato	evento que entra no fechamento por regra
Cota	unidade de peso do veículo no rateio
Competência	mês do fechamento em que o evento foi rateado
Dedução	linha que reduz o numerador (salvados, recuperação de inadimplentes)
Denominador	soma dos pesos de cota dos veículos elegíveis
D1 a D8	decisões do PRD da AVAP
Estabilização	contribuição para suavizar o valor da cota entre meses
Fila	eventos com `Valor Rateado = 0.00` no SGA, em preparação para o fechamento
Foto	exportação do SGA num momento, tratada como estática
Integral	indenização total (V.F.I, furto, roubo)
Invariante	propriedade que o resultado precisa satisfazer (ex.: alvo igual à soma alocada)
Lançamento	linha de soma ou dedução
Lacre	hash SHA-256 dos insumos de um fechamento
Numerador	custo dos eventos mais somas, menos deduções
Participação	parcela do evento paga pelo associado
Potinho	degrau cumulativo do ELLO
Protocolo	identificador do evento no SGA
Salvado	resto do veículo indenizado, vendido para abater custo
SGA	sistema de gestão da associação (Hinova)

Termo	Significado
SDD	Spec Driven Development
V.F.I	motivo de evento de perda integral

14. Onde ler mais

AVAP (nesta ordem):

1. `docs/REGRAS_RATEIO_AVAP.md`: **fonte única das regras**, com status e fonte de cada uma. Comece por aqui.
2. `docs/PRD_AVAP.md`: especificação técnica (leia com a seção 8.16 deste guia ao lado).
3. Histórico, só para entender como se chegou às regras: `docs/ANALISE_RATEIO_AGOSTO_SETEMBRO_AVAP.md`, `docs/LEVANTAMENTO_AVAP_organizado.md`, `docs/RELATORIO_ANALISE_AVAP_modelo_rateio.md`, `docs/AVAP_integrais_fora_do_rateio_ago_set_2026.md` e `docs/AVAP_eventos_retirados_fila_09_2026.md`.

AutoTruck: `docs/PRD_AutoTruck.md`, `docs/ACEITE_TECNICO_AutoTruck.md`, `docs/DICIONARIO_DADOS_AutoTruck.md`, `docs/ESPECIFICACAO_INSUMOS_RATEIO_AutoTruck.md`, `docs/RELATORIO_ADERENCIA_AutoTruck.md`, `docs/HOMOLOGACAO_carta.md`.

ELLO: `PRD.md`, `CONTEXTO.md`, `SETUP.md` (fase inicial), `docs/RELATORIO_ADERENCIA_ELLO_dados_reais.md`.

Regulatório e visão: `docs/Roadmap de Desenvolvimento de Software_ Plataforma para Administradoras (LC 213_25).md`, `docs/PLANO_REUNIAO_MUTUALIZA.md`, `docs/APPM_REDE_DISTRIBUICAO.md`.

Admissão Verificada: `admissao-verificada/PROJETO.md`, `specs/MVP-SCOPE.md`, `docs/adr/`.

Para entender o código AVAP em 30 minutos: leia `domain.py`, depois `engine.py` (76 linhas), depois `selecao.py`, depois rode `test_avap_fase1.py -v` e olhe os testes `test_real_*`.